

Agent Observability is Not Microservice Observability

Anonymous Author(s)

Submission anonymized for double-blind review

Abstract—Large language model (LLM) agents are failing in production in ways fifteen years of microservice observability research cannot represent. A coding agent on `django__django-10914` issues four near-identical search queries and exhausts its iteration budget; under the de-facto OpenTelemetry stack the failure surfaces as eight `INTERNAL` spans with no signal that the agent was stuck. Across a controlled 14-fault benchmark, vanilla OpenTelemetry and the current `gen_ai.*` semantic conventions saturate at a fault-detection rate of 0.429 (6 of 14); the missing eight are exactly the faults that arise from open-ended action, model-of-the-world reasoning, and emergent multi-agent coordination. We argue this is not a missing-attribute problem but a metamodel mismatch — the *Cognitive-Trace Hypothesis*: agent execution is a typed trajectory through a cognitive state space, not a request–response chain over RPC. From this reframing we derive five concrete research questions for the SE community, each anchored to a deliverable artifact. One — estimating what an autonomous agent would have done absent a runtime intervention — recasts SE evaluation as causal inference on a non-stationary stochastic policy, a class of empirical-SE question the community has not previously had to confront. The window to set these abstractions is short: production telemetry is already congealing around vendor-specific platforms.

Index Terms—LLM agents, observability, distributed tracing, OpenTelemetry, semantic conventions, fault detection, software engineering research agenda

I. INTRODUCTION

Vignette. A ReAct-style coding agent [1] attacks the SWE-bench Lite [2] instance `django__django-10914`. Over four consecutive iterations it issues near-identical calls to `search_code` with arguments `"FilePathField", "FilePathField", "FilePathField allow_files", "FilePathField"` — never converging, finally exhausting the eight-iteration budget without producing a patch. On the maintainer’s screen this run renders as eight `INTERNAL` OpenTelemetry spans of indistinguishable type. There is no signal that an agent reasoning loop occurred. The span graph satisfies the OpenTelemetry specification [3]; the agent silently failed.

Why the obvious vanilla-OTel detector does not work. A skeptic proposes a one-liner: hash the `gen_ai.completion` text on each `POST /v1/chat/completions` span and alarm on repetition. The proposal fails for two structural reasons. (i) The completion payload is a multi-thousand-token chain-of-thought blob fusing planning, tool-argument construction, and self-critique into one opaque string; the *step within* the chain is not addressable, so two “different” completions can encode the same underlying reasoning loop

and two “identical” completions can encode different cognitive states. (ii) Consecutive `chat/completions` calls in vanilla OTel carry no causal-edge type distinguishing “same agent looping” from “two independent agents querying the same backend”: the parent-child edge is the RPC parent, not the cognitive predecessor. The missing constructs — a typed cognitive predecessor edge plus a phase-addressable span — do not exist in the standard.

This is one trace among 84 such traces in a 112-instance study using a recently-released open-source agent-telemetry SDK [4]. The same SDK’s controlled benchmark shows that the gap is not an artifact of any one detector: vanilla OpenTelemetry and the `gen_ai.*` semantic conventions both saturate at a binary fault-detection rate of 0.429 (6 of 14 fault classes) on a controlled fault-injection harness [5]. The eight undetectable faults — planning failure, reasoning loop, guardrail bypass, circular delegation, memory corruption, stale retrieval, agent misroute, and hallucination unrelated to retrieval — have no representation in a metamodel whose root abstraction is a typed RPC span.

Thesis. Microservice observability assumes that an execution is a DAG of typed RPCs with bounded action spaces, no internal state worth recording, and a single causal call tree per request. None of those assumptions hold for LLM agents. The problem is not a missing attribute; the metamodel was designed in 2010 for a different class of system [6], [7]. We name the reframing the *Cognitive-Trace Hypothesis*: agent observability is fundamentally a typed trajectory through a *cognitive state* space, where *cognitive state* denotes the agent’s evolving model-of-the-world — planning context, intermediate reasoning chain, memory contents, guardrail decisions — updated at every internal step and not addressable through the request–response endpoint surface that microservice tracing assumes. The hypothesis is the third generation of a recurring SE move: distributed tracing extended single-process logging [6]; the hidden-technical-debt line extended software-systems monitoring for ML pipelines [8], [9]. Agents are the third inflection.

This paper does three things. (1) It identifies three structural mismatches between microservice tracing and agent execution (§II). (2) It reports one emerging empirical result that motivates the reframing (§III). (3) It articulates a research agenda of five concrete questions, each tied to a deliverable artifact (§IV), and identifies threats to the vision (§V, §VI). The deepest of those questions — estimating what an agent would have done absent a runtime intervention — recasts SE evaluation as

causal inference on a non-stationary stochastic policy, a class of empirical-SE question the community has not previously had to confront. The audience is the SE research community: the problem is one of abstraction, conformance, and contracts — not of training better models — and the window to set these abstractions before production telemetry congeals around vendor-specific platforms is now.

II. WHY MICROSERVICE OBSERVABILITY IS INSUFFICIENT

A. The microservice telemetry model

Dapper [6], X-Trace [10], Zipkin [11], Jaeger [12], and the OpenTelemetry consensus [3] model an execution as a DAG of typed spans (SERVER, CLIENT, INTERNAL, PRODUCER, CONSUMER) attached to fixed semantic-convention namespaces (`http.*`, `db.*`, `rpc.*`, `messaging.*`). The graph is *closed*: every node corresponds to a deployed binary’s RPC boundary. Tools downstream (latency dashboards, error-rate SLOs, trace-graph viewers) make this closure assumption explicit. The closure is what makes the metamodel *useful*: an SRE can map every span back to a service they can ssh into. Pivot Tracing [13] and the broader AIOps literature [14], [15] extend this picture with dynamic queries and learned anomaly detectors, but the underlying metamodel is unchanged.

B. Three structural mismatches

M1. The action space is open. A microservice has a fixed RPC surface; an agent’s next action is sampled from a learned distribution over tools, sub-agents, internal reasoning steps, and memory operations. The “`POST /v1/chat/completions`” span at the bottom of every agent trace conceals whether the request was a planning decomposition, a guardrail check, a chain-of-thought continuation, or a final answer. A `SpanKind` enum sized for transport semantics cannot represent cognition.

M2. Causal edges are RPC parent-child, not state propagation. Agent failure is dominated by state corruption that flows through a shared memory or context window, not through an HTTP header. When an agent writes a wrong fact to its memory store at iteration 3 and a later sub-agent retrieves it at iteration 7, no span-link in the existing OTel model encodes the dataflow. The `traceparent` W3C convention preserves causal containment but not causal data flow [3].

M3. Aggregations are per-endpoint, not per-decision. Microservice SLOs are formulated over $\langle \text{endpoint}, \text{status}, \text{latency} \rangle$ tuples. An agent fleet’s service-level concerns are over $\langle \text{task}, \text{recovery rate}, \text{cost-per-task}, \text{intervention-trigger-rate} \rangle$ tuples with no pre-defined endpoint. Practitioners currently set agent SLOs by analogy — “mean latency,” “error rate” — producing thresholds that are meaningless on a stochastic executor.

C. The current state of “LLM observability”

Commercial GenAI tracing stacks (LangSmith [16], Langfuse [17], OpenLLMetry [18], AgentOps [19]) intercept API calls and expose prompt/completion content. They are useful, but their underlying span model inherits the

microservice metamodel: a planning step and a summarization call are both `POST /v1/chat/completions`. As of the OpenTelemetry GenAI SIG draft conventions (v1.30, `gen_ai.*` namespace), the standardized vocabulary covers `gen_ai.system`, `gen_ai.request.model`, `gen_ai.completion`, `gen_ai.usage.*`, and the operation names `chat`, `execute_tool`, `retrieval` [20]. None of these typed kinds or attributes addresses the cognitive predecessor edge or the phase-within-completion addressability that the obvious-detector demolition in §III requires. OpenInference [21] goes furthest, defining typed `LLM`, `retriever`, `chain`, `agent`, and `guardrail` spans. It is the closest analogue to the proposal we sketch; the gap it leaves is, however, exactly the gap that matters: there is no typed distinction between a planning step and an interior reasoning step (both fall under `chain`); the `agent` span carries no source/target identifiers for typed inter-agent `DELEGATION`; and there is no `MEMORY` span kind for the read/write operations that drive most state-corruption faults. The OpenInference conventions are also not part of the OpenTelemetry standard. The OpenTelemetry GenAI SIG itself acknowledges fragmentation and an incomplete vocabulary as open issues [20]. Recent failure taxonomies for multi-agent systems characterize *what* fails but supply no runtime telemetry primitives capable of detecting those failures from standardized traces: MAST [22] catalogs 14 failure modes from manual transcript analysis but does not define detection signals; AgentDebug [23] learns from failure transcripts but assumes a richer trajectory representation than vanilla OTel exposes; Aegis [24] targets agent-environment mismatches but is content-oriented, not span-typed; AgentRx [25] diagnoses post-hoc from trajectories rather than from in-flight typed spans. AGDebugger [26] is a debugger that consumes such traces; without typed spans it cannot attribute the failure to a cognitive phase. Enforcement tools (NeMo Guardrails [27], GuardAgent [28]) are complementary — enforcement without typed monitoring leaves a blind spot.

III. EMERGING RESULT

We summarize *one* number that motivates the agenda; the underlying benchmark is reported in full elsewhere [5]. A 9-kind agent-telemetry SDK [4] was deployed across seven agent frameworks on 112 SWE-bench Lite instances under a ReAct coding-agent architecture [1]. The SDK records nine span kinds covering the orchestration phases absent from the OpenTelemetry GenAI conventions: `AGENT`, `LLM_CALL`, `TOOL_CALL`, `PLANNING`, `REASONING`, `RETRIEVAL`, `GUARD_RAIL`, `DELEGATION`, `MEMORY`.

Of 112 runs, 84 (75%) exhausted the iteration budget without producing a patch. All 84 exhibit a single structural pattern: three or more consecutive `REASONING` $\rightarrow$ `LLM_CALL` $\rightarrow$ `TOOL_CALL` cycles with near-identical tool arguments. The pattern is *only visible* once the trace carries a typed `REASONING` span kind; under vanilla OpenTelemetry the same 84 traces appear as approximately 28 undifferentiated `INTERNAL` spans each (3,060 spans total, no signal). A controlled fault-injection benchmark spanning 14 fault classes

confirms the gap is structural rather than threshold-tunable: vanilla OpenTelemetry and the current `gen_ai.*` conventions both saturate at a binary fault-detection rate of 0.429 (6 of 14); the eight undetectable faults are exactly those whose detection signal requires a span kind *not present* in the existing standard, so no detector built on the standard vocabulary in [5] was able to bridge the gap, independent of threshold choice. The number we draw on for this paper is the 75% silent-loop rate — not the full benchmark — because it is the minimum evidence the vision requires: *the dominant failure mode of a deployed coding agent is silent under the de-facto observability standard, and the silence is metamodel-level, not detector-level.*

IV. A RESEARCH AGENDA

Five open research questions, each anchored to a concrete artifact the SE community can build:

RQ1: Under what conditions is a span-kind vocabulary saturated, and what kinds of new agent architectures break a given vocabulary? The OpenTelemetry GenAI conventions today expose nine *operation names* (`chat`, `execute_tool`, `retrieval`, ...) but no typed *kinds* for the cognitive orchestration phases. The pilot taxonomy [5] derives nine kinds from seven open-source frameworks and reports saturation after five frameworks — but the question of how *principled* that saturation is remains open. Does a debate / tree-of-thought / self-consistency architecture require additional kinds (e.g., `CRITIC`, `VOTE`)? Should memory be one kind or one per backend (vector, KV, episodic)? How do we test, in advance, whether a candidate vocabulary will hold for the next architecture? *Artifact*: a community-curated taxonomy with explicit saturation criteria, shipped as an OpenTelemetry semantic-convention PR. *Tractable start*: a multi-coder open-coding study across ≥ 10 open-source agent frameworks with inter-coder reliability reporting, plus a stress test against emerging multi-agent architectures.

RQ2: How do we specify, at design time, what a correct agent trace should look like? Microservice OTel has no notion of an *expected* trace; the metamodel is descriptive. For stochastic agents we need a contract-style language — analogous to OCL for UML or trace-temporal-logic for distributed systems — typed over cognitive span kinds. Figure 1 sketches the form. The contract is not “this trace happened” but “every trace must look like this.” This is software contracts re-imagined for stochastic executors. *Artifact*: an expected-trace specification language, a reference checker, and a corpus of specifications mined from successful traces. *Tractable start*: mine `must-precede`, `must-contain`, and `must-not-repeat` patterns from the 28 successful SWE-bench runs (the complement of the 84 silent failures in §III) in [4] and adapt LTL_f for cognitive spans.

RQ3: How do we detect agent faults that are emergent — visible only in the trace as a whole, not in any single span? Reasoning loops, circular delegation, planning regression, and Byzantine sub-agents are trace-level phenomena. The pilot benchmark shows that an attribute-level detector cannot catch them even with the right schema; one needs trace-graph queries

```
spec ReActAgent {
  every PLANNING span p =>
    p :precedes (TOOL_CALL | DELEGATION)
      :before LLM_CALL;
  not exists consecutive REASONING r1, r2
    where hash(r1.tool_payload)
      == hash(r2.tool_payload);
  every MEMORY.write w =>
    exists MEMORY.read r in trace
      where r.key == w.key
        and r :after w;
}
```

Fig. 1. Sketch of an expected-trace specification (RQ2) over cognitive span kinds. Clause 1 catches *planning-failure* faults; clause 2 catches the *reasoning-loop* pattern from the emerging result (§III); clause 3 catches *memory-corruption* / orphan-write faults. Operators (`:precedes`, `:before`, `:after`, `exists in trace`) carry standard LTL_f -over-event-traces semantics typed over cognitive span kinds; full formalization is deferred to the artifact (RQ2). The language is intentionally open — a research artifact, not a finished product.

with cognitive typing. *Artifact*: a benchmark of trace-level fault detectors with ground-truth labels for at least the eight currently-undetectable faults, plus precision/recall reporting against a real-LLM trace corpus. *Tractable start*: extend the MAST behavioral taxonomy [22] with trace-level detection signals — the 12-of-14 MAST coverage already reported [5] is a starting point, not an end state.

RQ4: What is the right family of stochastic-SLO formalisms for non-stationary agent fleets, and how do we express them in telemetry? Latency and error-rate are the wrong primitives: they were designed for stationary request-response systems with bounded action spaces. Candidate primitives include task-recovery rate after intervention, cost-per-successful-task, plan-regression-rate (the fraction of plans superseded within k steps), and the intervention-trigger-rate of the safety policy — but the deeper open question is whether agent SLOs need a fundamentally different formalism (e.g., per-trajectory percentiles, drift-aware burn-rate windows) because the underlying agent policy is non-stationary across model updates and tool upgrades. *Artifact*: a catalog of agent-SLO patterns with PromQL/OTel-compatible queries, validated against at least one production agent fleet’s traces, paired with a formal characterization of which patterns are robust under policy drift. *Tractable start*: formalize the four metrics above for the pilot corpus and report their stability across model tiers.

RQ5: How do we close the loop — from observability to runtime intervention — without violating the principle that telemetry is side-effect-free? Production agent platforms increasingly do this in an ad-hoc way: detect a cost explosion, switch to a cheaper model; detect a reasoning loop, inject a strategy-change prompt. The pilot work [5] reports a small +12.5pp recovery gain from one such intervention at $n=24$ (not statistically significant; significance is reserved for later work). Treating span processors as first-class *actuators* re-opens questions SE thought it had solved: how to record the intervention as a causally-linked event [10]; how to prevent intervention loops; and — the deepest of the three — how

to estimate the counterfactual: what would the agent have done absent the intervention? That is, in effect, a causal-inference problem on a non-stationary stochastic policy, a class of empirical-software-engineering question the community has not previously had to confront. *Artifact*: a research agenda for intervention-aware tracing, with formal semantics for actuator-spans and a causal-inference methodology for evaluating closed-loop interventions in non-stationary agent populations.

Together these five questions cut across at least four core ICSE research areas (observability, testing, formal methods, debugging) and form a research program. None of them require advances in model training; all of them are squarely in the SE community’s purview — abstractions, conformance, and contracts.

V. RELATED VISION WORK

Position papers on AI for SE and SE for AI have laid groundwork on quality assurance for ML systems, on *using* LLMs as SE tools [29], and on the methodological threats of LLM-augmented research [30]; recent visions of AI-native software engineering call for re-imagining the SE lifecycle around foundation models [31]. The structurally closest precedent is the *ML-systems observability* line: Sculley et al.’s hidden-technical-debt argument [8] and the ML Test Score [9] made the case a decade ago that ML pipelines exhibit qualitatively new observability requirements (training-serving skew, feedback loops, entanglement). The Cognitive-Trace Hypothesis is the third generation of that move: distributed tracing extended single-process logging [6], [7]; ML observability extended distributed tracing; agent observability extends ML observability with open-ended action spaces and intra-trace cognitive-state evolution.

Within the agent ecosystem, failure taxonomies [22]–[24] and debugging tools [25], [26] make the empirical case that agent failures are dominated by cognitive and coordination pathologies; they do not position the gap as a metamodel problem for the SE community. The SDK [4] on which our emerging result rests proposes typed cognitive span kinds, but presents them as a benchmark contribution, not as the seed of a research agenda. The contribution of the present paper is the agenda framing.

VI. THREATS TO THE VISION

T1: A narrower vocabulary wins. The field may converge on an LLM-only vocabulary that collapses agent semantics into tool-call sequences. The emerging result already shows this is structurally insufficient: tool-call sequences alone cannot distinguish a reasoning loop from a planned multi-step search.

T2: Frameworks consolidate. If agent frameworks consolidate around a single dominant API, cross-framework abstractions may seem unnecessary. The pilot data shows that *even within a single framework* (LangChain on the 112-instance SWE-bench corpus in [4]) the structural mismatches (M1–M3) hold.

T3: Vendor capture. Production telemetry may shift toward proprietary single-vendor platforms (Datadog LLM, Anthropic-internal, OpenAI Traces) that absorb the standard. SE research

should pre-empt this by establishing open, typed semantics now — the same role the Dapper paper [6] played for microservices.

T4: Chain-of-thought unfaithfulness. The model’s verbal reasoning chain need not be a faithful record of its internal computation [32]; if a REASONING span’s payload is the recorded CoT, an unfaithful CoT yields a misleading span. The agenda does not collapse under this threat because cognitive span kinds separate cognitive *phases* from cognitive *contents*: the phase-level structural faults that the emerging result targets (loops, missing planning, orphan memory writes) are detectable from phase typing and causal-edge structure alone, independent of CoT content faithfulness. Faithfulness becomes a sub-question for the contents of typed spans (an RQ2 specification can constrain phases without trusting CoT verbatim), not a refutation of the metamodel.

VII. FUTURE PLANS

The agenda is multi-year. Concrete near-term steps planned by the authors and proposed for collaboration with the community:

- 1) **Full ICSE Research-Track benchmark** extending the 14-fault controlled study to additional architectures (tree-of-thought, debate, hierarchical multi-agent) and a ≥ 500 -instance real-LLM corpus.
- 2) **Semantic-convention proposal** of typed cognitive span kinds to the OpenTelemetry GenAI SIG, with a public PR (citation withheld for double-anonymous review) and a reference implementation across at least three agent frameworks. The lived experience of the upstream review process is itself empirical input for RQ1.
- 3) **Expected-trace specification language** (RQ2), released as a stand-alone artifact with a checker and a mined-spec corpus.
- 4) **Controlled human study** ($n \approx 30$ SE practitioners) measuring time-to-diagnose with versus without typed cognitive spans, with IRB-equivalent informed consent and pre-registered analysis.
- 5) **Agent-SLO pattern catalog** (RQ4), validated against a production agent fleet’s telemetry, released as an OpenTelemetry recipe collection.

Anonymized artifacts (the SDK source, the SWE-bench trace corpus, the fault-injection harness, the semantic-convention specification) will be released with the camera-ready version under an open license.

VIII. CONCLUSION

Microservice observability is the wrong frame for autonomous agent execution: the metamodel was designed in 2010 for typed RPC over fixed action spaces. The Cognitive-Trace Hypothesis recasts agent observability as typed trajectories through a cognitive state space; the 75% silent-loop rate under vanilla OpenTelemetry establishes the gap as structural. Five research questions, each anchored to a concrete artifact, define a tractable agenda — one (RQ5) opens causal inference on non-stationary stochastic policies as a new class of empirical-SE question. The window to set these abstractions is short.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023.
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [3] OpenTelemetry Authors, "OpenTelemetry specification (v1.30)," <https://opentelemetry.io/docs/specs/otel/>, 2024.
- [4] Anonymous Authors, "AgentTelemetry: An open-source agent-observability SDK (citation withheld for double-anonymous review)," 2026, under review; reference withheld for double-anonymous review.
- [5] —, "A fault-detection benchmark and span taxonomy for LLM agent observability (citation withheld for double-anonymous review)," in *Proceedings of an SE venue (anonymized)*, 2026, under review; reference withheld for double-anonymous review.
- [6] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspán, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure," in *Google Technical Report*, 2010. [Online]. Available: <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>
- [7] A. Parker, D. Spoonhower, J. Mace, B. Sigelman, and R. Isaacs, *Distributed Tracing in Practice: Instrumenting, Analyzing, and Debugging Microservices*. O'Reilly Media, 2020.
- [8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [9] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *IEEE International Conference on Big Data*, 2017.
- [10] R. Fonseca, G. Porter, R. H. Katz, S. Shenker, and I. Stoica, "X-Trace: A pervasive network tracing framework," in *NSDI*, 2007.
- [11] The OpenZipkin Authors, "Zipkin: A distributed tracing system," <https://zipkin.io/>, 2024.
- [12] The Jaeger Authors, "Jaeger: Open-source, end-to-end distributed tracing," <https://www.jaegertracing.io/>, 2024.
- [13] J. Mace, R. Roelke, and R. Fonseca, "Pivot Tracing: Dynamic causal monitoring for distributed systems," in *SOSP*, 2015.
- [14] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Transactions on Intelligent Systems and Technology*, 2021.
- [15] S. Karumuri, F. Solleza, S. Zdonik, and N. Tatbul, "Towards observability data management at scale," in *SIGMOD Record*, 2021.
- [16] LangChain Inc., "LangSmith: Production-grade LLM platform," <https://smith.langchain.com/>, 2024.
- [17] Langfuse GmbH, "Langfuse: Open-source LLM observability," <https://langfuse.com/>, 2024.
- [18] Traceloop, "OpenLLMetry: Open-source LLM observability," <https://github.com/traceloop/openllmetry>, 2024.
- [19] L. Dong, Q. Lu, and L. Zhu, "AgentOps: Enabling observability of LLM agents," 2024.
- [20] OpenTelemetry Authors, "AI agent observability: Evolving standards and best practices," <https://opentelemetry.io/blog/2025/ai-agent-observability/>, 2025.
- [21] Arize AI, "OpenInference: Open-source specification for AI observability," <https://github.com/Arize-ai/openinference>, 2024.
- [22] M. Cemri, M. Z. Pan, S. Yang *et al.*, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [23] K. Zhu, Z. Liu, B. Li *et al.*, "Where LLM agents fail and how they can learn from failures," 2025.
- [24] K. Song, A. Jayarajan, Y. Ding *et al.*, "Aegis: Taxonomy and optimizations for overcoming agent-environment failures in LLM agents," 2025.
- [25] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal, "AgentRx: Diagnosing AI agent failures from execution trajectories," 2026.
- [26] W. Epperson, G. Bansal, V. Dibia, A. Fournery, J. Gerrits, E. Zhu, and S. Amershi, "Interactive debugging and steering of multi-agent AI systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI)*, 2025.
- [27] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *EMNLP System Demonstrations*, 2023.
- [28] Z. Xiang, L. Zheng, Y. Li *et al.*, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [29] D. Lo, "Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps," in *Proceedings of the IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 2023.
- [30] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: the threats of using LLMs in software engineering," in *Proceedings of the 46th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2024.
- [31] A. E. Hassan, D. Lin, A. Nikanjam, F. Khomh, K. Lakkaraju, and B. Adams, "Towards AI-Native software engineering (SE 3.0): A vision and a challenge roadmap," in *Proceedings of the 1st International Workshop on Vision of Foundation Models in Software Engineering*, 2024.
- [32] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.